Reconstructing Hardware Wallet Exploits

Mike Tolkachev / Specter Association

EDUCATIONAL USE DISCLAIMER

Authorized activity: interact with weaknesses only in the workshop devices.

Limited scope: applies solely to this workshop.

Workshop equipment: devices may be modified, stressed, or disabled.

Personal system responsibility: you're responsible for your laptop and data.

Ethical & lawful use: apply knowledge responsibly and legally.

Acceptance of terms: participation constitutes acknowledgment and acceptance.

Why exploits, not broken crypto?

- Wallet failures are often **system design / implementation / assumptions**
- Security boundaries leak through:
  - variable-time behavior
  - malleable length fields
  - memory corruption
  - unchecked parsing assumptions
  - implicit trust in external inputs
- We'll reconstruct a real-world style **bug chain** on a safe target

Today's target: the toy wallet

- RP2040 (Raspberry Pi Pico)
- Two USB serial (CDC) interfaces
- Deliberately vulnerable firmware for education

All lab steps/commands live in `labs/*.md`.

Protocol

- Composite USB device with **two CDC ACM** interfaces
- **CDC 0 (UX/log)**: UI frames, logs, and **seed backup output**
- **CDC 1 (protocol)**: machine-driven request/response
- Line format: 1 line per message: `<opcode>[ <hexarg>]\n`
  - ASCII opcodes (short), optional **even-length hex** argument
  - Responses: `ok\n`, `err\n` or `ok <hex>\n`
- State gating tries to be strict

Commands

Command	Request	Response	Notes
`ping`	`ping`	`ok`	liveness check
`lock`	`lock`	`ok` / `err`	locks device
`pin`	`pin <pin_hex>`	`ok` / `err`	unlocks on success
🔒 `state`	`state`	`ok <01 byte hex>`	returns state byte
🔒 `info`	`info` or `info <len_hex>`	`ok <hex_blob>` / `err`	returns bytes from `wallet_info_t`
🔒 `pass`	`pass <pass_hex>`	`ok` / `err`	set passphrase
🌟 `backup`	`backup`	`ok` / `err`	displays mnemonic (onboarding)

Example session

PIN: 12345

Requested info: first 16 bytes of `wallet_info_t`

```
> ping
< ok
> info 0010
< err
> pin 0102030405
< ok
> info 0010
< ok 544f5957414c4c4554000000000000
> backup
< err
```

Exploit chain: Lab 1 → Lab 2 → Lab 3

1. Side-channel → recover PIN
2. Over-read → leak internal pointer
3. Stack overflow → hijack return address (“ret2win”)

Bonus for champions - Lab 4

4. ROP → code reuse with attacker-controlled args

Lab 1: Concept

Timing side-channel on PIN verification

Even with only `ok` / `err`, **time can leak secrets.**

```
bool pin_ok(const uint8_t *guess, const uint8_t *pin, size_t n) {
  for (size_t i = 0; i < n; i++) {
    if (guess[i] != pin[i]) return false;  // early exit
    sleep_ms(200);                         // measurable leak
  }
  return true;
}
```

This is how `memcmp` actually implemented internally in most cases.

Lab 1: Historical anchor

Side-channels are “physics bugs”

- Variable-time or secret-dependent behavior can leak via:
  - wall-clock timing (protocol/USB)
  - **power analysis**
  - **electromagnetic** emanations

Real-world examples

- **CVE-2013-0169 “Lucky Thirteen”** — Plaintext recovery via timing side-channel in TLS CBC padding/MAC.
- **T-wallet power analysis (Hoenicke, 2015)** — Power traces were used to recover secrets.
- **CVE-2017-5753 “Spectre v1”** — Speculative execution leaks secrets through cache timing.
- **L-wallet (wallet.fail, 2018)** — Recovering PIN from touchscreen EMI via SDR receiver.
- **CVE-2019-14353…-14357** — Multiple wallets with OLEDs can leak PIN/seed via USB power fluctuations.

Lab 1: Mitigation

Constant-time primitives + defensive UX

- Constant-time comparisons for fixed-length secrets
- Avoid secret-dependent branching in auth paths (including error paths)
- Attempt limits + backoff help, but **don't replace** constant-time code
- Ensure no correlation in power traces via masking and noisy peripherals/cores

Lab 2: Concept

Over-read → memory disclosure → pointer leak

Client-controlled lengths are dangerous.

```
bool cmd_info(size_t requested_len) {
  const uint8_t *base = (const uint8_t *)&wallet_info; // adjacent data follows
  // BUG: requested_len not bounded to sizeof(wallet_info)
  send_hex(base, requested_len);
  return true;
}
```

Lab 2: Historical anchor

“Length field lies” is a classic pattern

- Protocol says: “here’s my length”
- Implementation says: “sure, I’ll copy that many bytes”
- Result: **out-of-bounds read**, leaking adjacent memory

Real-world examples

- **CVE-2014-0160 “Heartbleed”** — Missing bounds check allows reading arbitrary process memory over net.
- **CVE-2020-6861, L-wallet** — Overread/oracle attacks allow extraction of Monero master spending key.
- **T-wallet leak via USB descriptor (O’Flynn, 2019)** — Fault injection + malformed USB requests could bypass a length check and leak adjacent data.
- **B-wallet response leak (Reitter, 2019)** — Few unintended bytes were included in a handshake response.

Lab 2: Mitigation

Boundaries + serialization discipline

- Bounds-check lengths against the actual object size
- Reject unexpected sizes (don’t silently clamp unless you have a reason)
- Don’t serialize raw structs directly; define explicit response fields
- Keep sensitive data **not adjacent** to USB descriptors
- Harden critical checks against fault injection

Lab 3: Concept

Stack overflow → return-address hijack (ret2win)

Decode/parse bugs often create write-what-where on the stack.

```
bool decode_hex(const char *hex, size_t hex_len,
                uint8_t *out, size_t cap) {
  // BUG: missing (hex_len/2 <= cap) check
  for (size_t i = 0; i < hex_len / 2; i++) out[i] = byte(hex, i);
  return true;
}

bool cmd_pass(const char *arg, size_t arg_len) {
  uint8_t decoded[64];                    // stack buffer
  decode_hex(arg, arg_len, decoded, sizeof(decoded));
  return true;                            // return address lives “nearby”
}
```

Lab 3: Historical anchor

Parsing is where “wallets meet attackers”

- Wallet firmware parses attacker-controlled inputs:
  - transactions / scripts
  - host-to-device messages
  - file-like USB features
- Memory corruption turns “input handling” into “control-flow control”

Real-world examples

- **T-wallet “malicious ScriptSig” (Bacca, 2014)** — Crafted transaction create a buffer overflow, enabling RCE.
- **CVE-2015-7547 “getaddrinfo”** — Crafted DNS responses overflow buffer in the resolver with RCE potential.
- **CVE-2019-0708 “BlueKeep”** — Crafted RDP requests could lead to RCE in Remote Desktop Services.
- **CVE-2021-31616, CVE-2023-27892** — Crafted messages allow RCE on K-wallet in different apps.

Lab 3: Mitigation

Stop corruption early

- Check bounds in the *decoder* (where bytes are written)
- Prefer safe parsing patterns (length-first, cap-enforced, fail-closed)
- Reduce stack exposure for untrusted payloads
- Use hardening where available (stack protections, W^X/NX-like where possible)
- Architect for containment (least privilege, MPU, isolate secrets)

Stack overflow: hang → ret2win → ROP

- A stack overflow is not “just a crash”:
  - crash/reset is a common outcome
  - but **control-flow corruption** can escalate to **RCE**
- “Signed firmware” and “attestation passed” do **not** prevent exploitation:
  - attackers reuse **your already-signed code**
  - they change *where execution goes*, not *what’s flashed*
- Trusted code ≠ trusted execution
  - memory safety bugs break assumptions about control flow
  - **ret2win**: jump to an existing “privileged” function
  - **ROP**: chain existing code gadgets to run attacker-chosen logic

Lab 4 (bonus)

ROP on microcontrollers (code reuse)

- “No injected code” doesn’t stop exploitation if you can control return flow
- ROP = reuse existing instructions (“gadgets”) to:
  - load registers with attacker-chosen values
  - call an existing function with attacker-chosen arguments
- On Thumb/Cortex-M, gadgets often look like `pop {..., pc}`

Practical part

https://github.com/miketlk/toy-wallet-workshop

- Scan the QR code and clone the repo
- Read `README.md` and follow setup instructions
- Run `client/ux.py` and observe the output
- Proceed with `labs/lab*.md` to solve the challenges

Final takeaway

If you can measure it, you can study it.

If you can abuse it, you can weaponize it.

If you can overwrite it, you can own it.